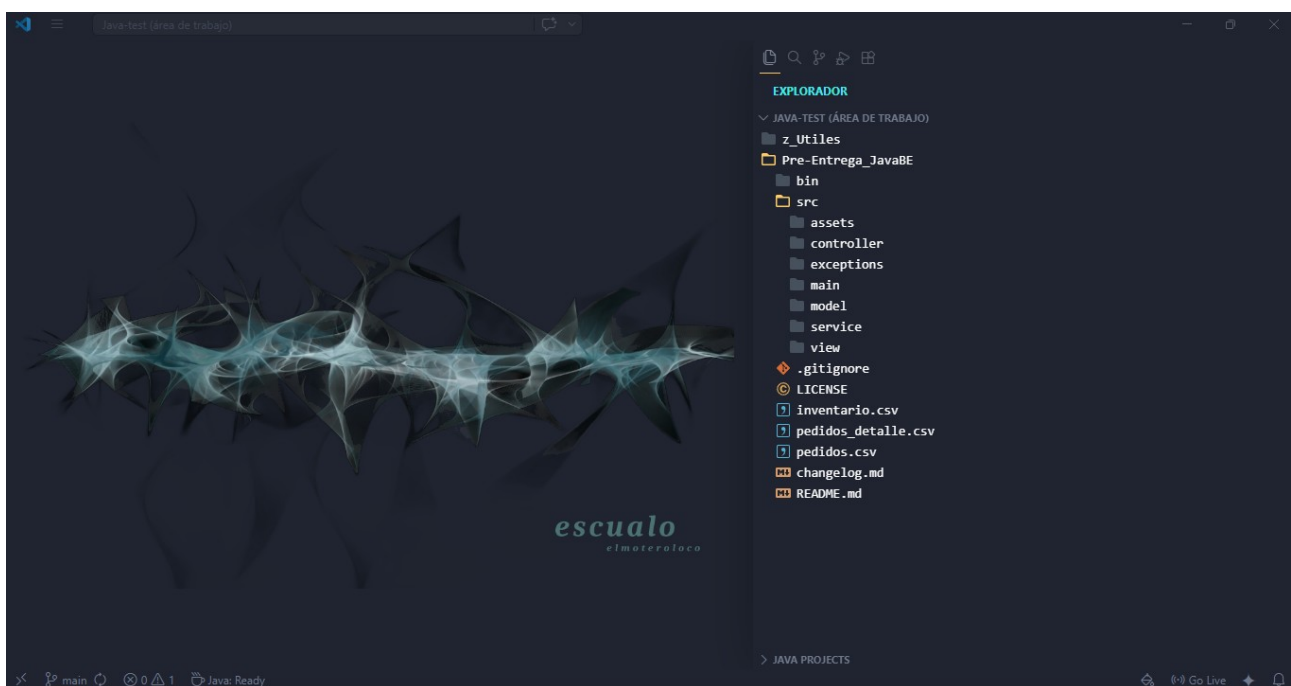


Elaboración del Proyecto para la Pre-Entrega

Enlace: [Pre-Entrega Java-BE en GitHub](#)

Empecé con lo básico (Producto, Main) y seguí con los ID "autoincrementales", validaciones de stock y un menú interactivo con While/Switch.

Investigué y agregué la persistencia con archivos .csv. También busqué una organización de arquitectura moderna (que me resultó cercana a como se usan en Node y React) (model, service, exceptions), que mas adelante terminó encajando bastante bien en un diseño "Controller/View".



Estructura final de directorios del proyecto

Creé la clase Bebida para implementar "herencia" y usé atributos "protected" para que las clases hijas los vean. Agregué el módulo de Pedido con los cálculos de descuentos automatizados (10% y 20%) y excepciones personalizadas para "falta de stock", mas adelante investigue y monté una "jerarquía" de validación mediante excepciones que también implementan un sistema de herencias y usan una superclase genérica.

```
Bebida.java X
13     this.esAlcoholica = esAlcoholica;
14 }
15
16 public Bebida(int id, String nombre, double precio, int stock, boolean e
17     super(id, nombre, precio, stock);
18     this.esAlcoholica = esAlcoholica;
19 }
20
21 public boolean isEsAlcoholica() {
22     return esAlcoholica;
23 }
24
25 @Override
26 public String getDetalleTipo() {
27     return esAlcoholica ? "Bebida (Alc)" : "Bebida";
28 }
29 }
30
```

Bebida.java

```
public Pedido(int id, double total, double totalSinDescuento, EstadoPedido estado) {
    this.idPedido = id;
    this.lineas = new ArrayList<>();
    this.total = total;
    this.totalSinDescuento = totalSinDescuento;
    this.estado = estado;
}

public void agregarProducto(Producto producto, int cantidad, double precioAplicado) {
    this.lineas.add(new LineaPedido(producto, cantidad, precioAplicado));
    this.totalSinDescuento += (producto.getPrecio() * cantidad);
    this.total += (precioAplicado * cantidad);
}

public static void setContadorPedido(int nuevoValor) {
    contadorPedido = nuevoValor;
}
```

Pedido.java (fracción)

```

Pedido.java X
52     public double getTotal() {
53         return total;
54     }
55
56     public double getTotalSinDescuento() {
57         return totalSinDescuento;
58     }
59
60     public double getAhorroTotal() {
61         return totalSinDescuento - total;
62     }
63
64     public List<LineaPedido> getLineas() {
65         return lineas;
66     }
67
68     public EstadoPedido getEstado() {
69         return estado;
70     }
71
72     public void setEstado(EstadoPedido estado) {
73         this.estado = estado;
74     }

```

Pedido.java (fracción)

Use Enums para manejar los estados de Pedidos (Pendiente, Entregado, Cancelado) aplicando el principio de "Polimorfismo". Esto me permitió armar un sistema de "Caja" realista que, al cancelár un pedido, activa la mecánica que devuelve el stock al inventario.

```

EstadoPedido.java X
1  package model;
2
3  public enum EstadoPedido {
4      PENDIENTE,
5      ENTREGADO,
6      CANCELADO
7  }

```

Uso de "enum"

Implemente el "borrado lógico" para no romper los históricos de ventas al borrar un producto. También los colores ANSI, opciones para cancelar operaciones a mitad de camino y otras mejoras en el Menú de consola para una interacción amigable.

```
public void eliminarProducto(List<Producto> inventario, int id) {
    Producto p = buscarProducto(inventario, id);
    if (p == null) throw new ValidacionProductoException("No se encontro el ID " + id);
    p.setActivo(activo: false);
}

public void reactivarProducto(List<Producto> inventario, int id) {
    Producto p = buscarProducto(inventario, id);
    if (p == null || p.isActivo()) throw new ValidacionProductoException(mensaje: "No se encontro un producto inactivo con ese ID.");
    p.setActivo(activo: true);
}
```

“Borrado lógico”

ID	Nombre	Precio	Stock
1	Cafe [Cafetería]	\$350,00	540
2	Te [Cafetería]	\$120,00	120
3	Yerba [Cafetería]	\$200,00	480
4	Chocolate [Cafetería]	\$300,00	340
5	Jerez [Bebida (Alc)]	\$500,00	450
6	Guindado [Bebida (Alc)]	\$300,00	360
7	Crema En Polvo [Cafetería]	\$200,00	600
8	Cognac [Bebida (Alc)]	\$800,00	150
9	Azucar [Cafetería]	\$150,00	100
10	Edulcorante [Cafetería]	\$150,00	100
11	Scotch [Bebida (Alc)]	\$1700,00	250
12	Capsulas De Cafe [Cafetería]	\$750,00	360
Nombre ('x' para cancelar): Amontillado			
Precio: 500			
Stock: 120			
¿Es una bebida alcohólica? (s/n): s			
Producto agregado correctamente.			

Mejora de UX/UI en consola (en la vista, 'x' para cancelar y color ANSI para operación exitosa)

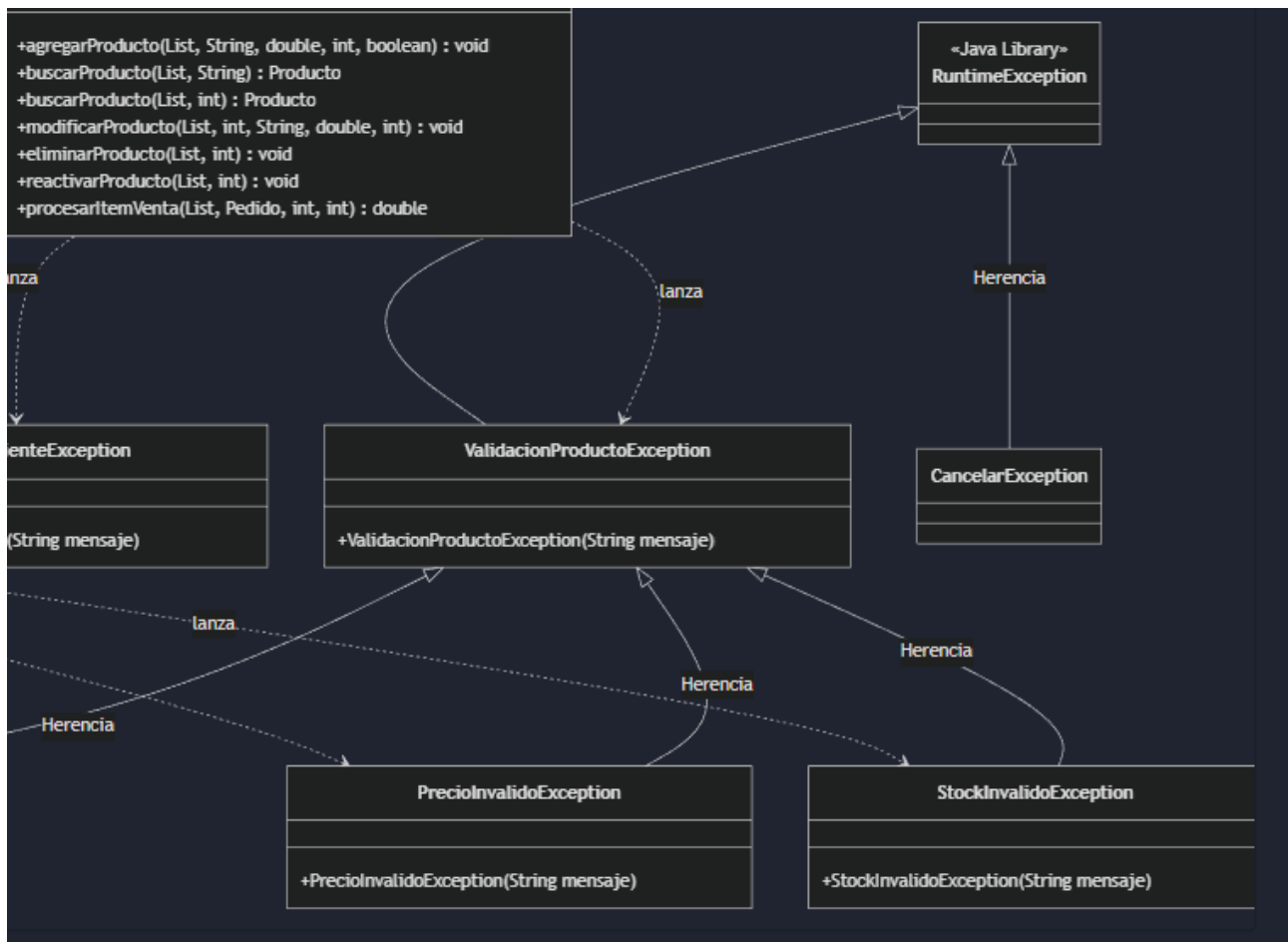
Apliqué "Responsabilidad Única" en un patrón "Controller/View" que agrega una jerarquía de excepciones que hace uso del principio de "herencia".

```
public void mostrarMenuPrincipal() {
    System.out.println(x: "\n— Menú de Inventario —");
    System.out.println(x: "1. Listar Productos");
    System.out.println(x: "2. Agregar Producto");
    System.out.println(x: "3. Buscar Producto");
    System.out.println(x: "4. Modificar Producto");
    System.out.println(x: "5. Eliminar Producto");
    System.out.println(x: "6. Reactivar Producto");
    System.out.println(x: "7. Realizar Pedido (Venta)");
    System.out.println(x: "8. Listar Pedidos Realizados");
    System.out.println(x: "9. Actualizar Estado de Pedido");
    System.out.println(x: "10. Reporte de Ventas");
    System.out.println(x: "...");
    System.out.println(x: "0. Salir");
    System.out.print(s: "Seleccioná una opción: ");
}
```

ConsoleView.java (fragmento)

```
private void procesarOpcion(int opcion) {
    switch (opcion) {
        case 1 → view.mostrarCatalogo(inventario);
        case 2 → gestionarAltaProducto();
        case 3 → gestionarBusqueda();
        case 4 → gestionarModificacion();
        case 5 → gestionarBajaProducto();
        case 6 → gestionarReactivacion();
        case 7 → gestionarVenta();
        case 8 → view.mostrarHistorialPedidos(pedidoService.getHistorialPedidos());
        ...
        case 9 → gestionarCambioEstado();
        case 10 → {
            double[] datos = pedidoService.calcularDatosReporte();
            view.mostrarReporte(datos[0], datos[1], (int) datos[2]);
        }
    }
}
```

MenuController (fragmento)



Jerarquia de excepciones con implementación de Herencia (Diagrama de Clases)

Modularicé la persistencia y los "servicios de pedidos".

```

PersistenceService.java X
1  package service;
2
3  import model.*;
4  import java.io.*;
5  import java.nio.charset.StandardCharsets;
6  import java.nio.file.*;
7  import java.util.List;
8
9  public class PersistenceService {
10     private static final String RED = "\u001B[31m";
11     private static final String RESET = "\u001B[0m";
12
13     public void guardarInventarioCSV(List<Producto> inventario) {
14         String ruta = "inventario.csv";
15         try (PrintWriter writer = new PrintWriter(Files.newBufferedWriter(Paths.get(ruta), StandardCharsets.UTF_8))) {
16             for (Producto p : inventario) {
17                 boolean esAlc = (p instanceof Bebida b) && b.isEsAlcoholica();
18                 writer.println(p.getId() + "," + p.getNombre() + "," + p.getPrecio() + "," + p.getStock() + "," + esAlc
19                     + "," + p.isActivo());
20             }
21         } catch (IOException e) {
22             System.out.println(RED + "Hubo un error al guardar el archivo de inventario: " + e.getMessage() + RESET);
23         }
24     }
25 }

```

PersistenceService.java (fragmento)

```

1 package service;
2
3 import model.*;
4 import exceptions.*;
5 import java.util.ArrayList;
6 import java.util.List;
7
8 public class PedidoService {
9     private ArrayList<Pedido> historialPedidos = new ArrayList<>();
10
11     public ArrayList<Pedido> getHistorialPedidos() {
12         return historialPedidos;
13     }
14
15     public void agregarPedidoAlHistorial(Pedido pedido) {
16         historialPedidos.add(pedido);
17     }
18
19     public void actualizarEstadoPedido(List<Producto> inventario, int idPedido, EstadoPedido nuevoEstado) {
20         Pedido pedidoEncontrado = historialPedidos.stream()
21             .filter(p -> p.getIdPedido() == idPedido)
22             .findFirst()
23             .orElseThrow(() -> new ValidacionProductoException("No se encontro el pedido con ID " + idPedido));
24     }

```

PedidosService.java

Hice tests asistidos por IA y agrego excepciones para casos puntuales como el de inventario vacío o fallos en los CSV.

```

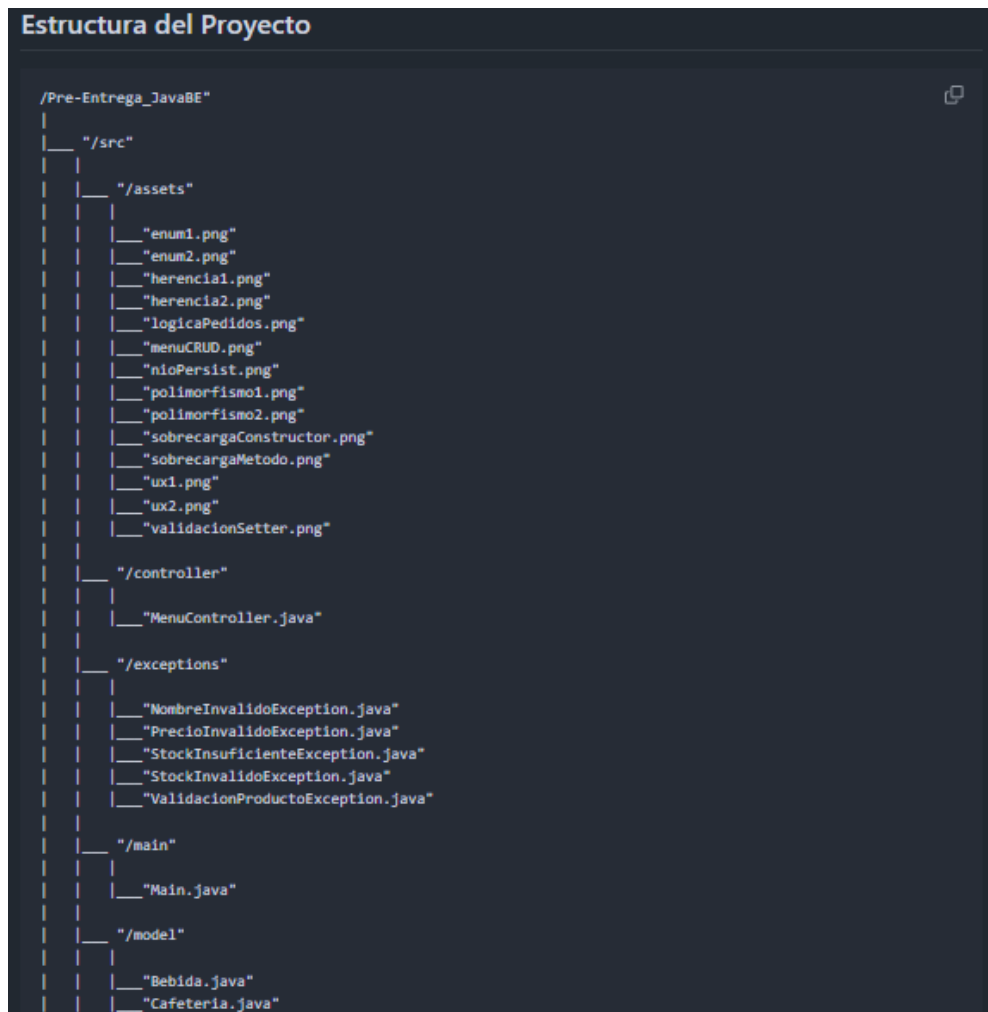
public void guardarPedidosCSV(List<Pedido> historial) {
    try (PrintWriter writer = new PrintWriter(Files.newBufferedWriter(Paths.get(first: "pedidos.csv"),
        StandardCharsets.UTF_8))) {
        for (Pedido p : historial) {
            writer.println(p.getIdPedido() + "," + p.getTotal() + "," + p.getTotalSinDescuento() + "," + p.getEstado());
        }
    } catch (IOException e) {
        System.out.println(RED + "Hubo un error al guardar el archivo de pedidos: " + e.getMessage() + RESET);
    }

    try (PrintWriter writer = new PrintWriter(Files.newBufferedWriter(Paths.get(first: "pedidos_detalle.csv"),
        StandardCharsets.UTF_8))) {
        for (Pedido p : historial) {
            for (LineaPedido lp : p.getLineas()) {
                writer.println(p.getIdPedido() + "," + lp.getProducto().getId() + "," + lp.getCantidad() + "," + lp.getPrecioAplicado());
            }
        }
    } catch (IOException e) {
        System.out.println(RED + "Hubo un error al guardar el archivo de detalles del pedido." + RESET);
    }
}

```

Manejo de excepciones en PersistenceService.java con Try/Catch

Creé un Readme.md incluyendo imagenes, un diagrama de clases en "Mermaid", un plano de la estructura, un enlace al "changelog.md" y 'destacados' usando el Markdown de GitHub.



Estructura del Proyecto (fragmento)

Para cubrir la inclusión de clases abstractas creé una nueva clase Cafeteria.java dejando a Producto con el modificador "abstract".

```
Producto.java X
1 package model;
2
3 import exceptions.*;
4 import java.util.Objects;
5
6 public abstract class Producto {
7     private static int contadorId = 0;
8     protected int id;
9     protected String nombre;
10    protected double precio;
11    protected int stock;
12    protected boolean activo = true;
```

Abstract en Producto.java